A MINOR PROJECT REPORT

on

HOSTEL TRACKER

A Cloud-Based Complaint Tracking and Maintenance Management System for University Hostels

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology / Master of Computer Applications

in Computer Science & Engineering

Submitted by

[Aryan Prajapati]

[25MCC20067]

Under the guidance of

[Dhruv Shah], [Ass. Professor]

Department of Computer Science & Engineering

[Chandigarh University / UIC]

Academic Session 2026–27

CERTIFICATE

This is to certify that the minor project report entitled “Hostel Tracker: A Cloud-Based Complaint Tracking and Maintenance Management System for University Hostels”, submitted by [Student Name], Roll No. [25MCC20067], is a bona fide record of work carried out under my supervision and guidance, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology / Master of Computer Applications in Computer Science & Engineering.

The work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

[Mr Dhruv Shah]
Project Guide

DECLARATION

I hereby declare that the minor project report titled “Hostel Tracker” is my own original work. The design, implementation and documentation were carried out by me under the guidance of my project supervisor. All external libraries, tools, books and online resources used during development have been duly acknowledged in the References section.

[Aryan Prajapati]

Roll No. [25mcc20067]

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, [Mr. Dhruv Shah], for the continuous guidance, encouragement and constructive feedback that shaped this project. I am also thankful to the Head of the Department and the faculty members of the Department of Computer Science & Engineering for providing the facilities and an environment that supported this work.

I extend my thanks to the hostel wardens, maintenance staff and fellow residents whose everyday experience with maintenance problems helped me understand the real requirements of the system. Finally, I am grateful to the open-source community behind React, Node.js, Express, PostgreSQL and Vite, whose tools formed the foundation of this project, and to my family and friends for their constant support.

[Aryan Prajapati]

ABSTRACT

Maintaining residential infrastructure in university hostels is an everyday operational challenge. Most hostels still depend on paper registers or informal messaging groups to record faults such as leaking taps, sparking sockets, broken locks or failed internet connections. These methods delay reporting, give students no visibility of progress, leave no reliable record of who did what, and make it impossible to measure how well maintenance is performing.

This project presents Hostel Tracker, a web-based complaint tracking and maintenance management system. Residents register and log complaints in a few steps; wardens assign them to maintenance staff; technicians update progress; and residents rate the completed work. The system is built as a three-tier application: a React 19 single-page application (built with Vite) on the front end, a stateless Node.js/Express REST API in the middle, and a Neon serverless PostgreSQL database for storage. Access is controlled by role (resident, staff, admin) using JSON Web Tokens, and passwords are hashed with bcrypt.

Key features include year-prefixed sequential complaint numbers a finite-state lifecycle for complaints, an audit history that records every change, staff assignment with workload visibility, a notice board with automatic expiry, resident star ratings, and an analytics dashboard that shows complaint counts, status breakdown, average rating and average resolution time. The application is deployed on Vercel (front end), Render (back end) and Neon (database), all on free tiers. The report documents the requirements, design, implementation, testing and results of the system, with screenshots of the working application.

Keywords: *Hostel maintenance, complaint management, React, Node.js, Express, PostgreSQL, JWT, role-based access control, audit trail.*

TABLE OF CONTENTS

Certificate 2

Declaration 3

Acknowledgement 4

Abstract 5

List of Abbreviations 7

Chapter 1: Introduction 8

Chapter 2: Literature Survey and Existing Systems 10

Chapter 3: Requirements Specification 12

Chapter 4: System Design 14

Chapter 5: Implementation 17

Chapter 6: REST API 20

Chapter 7: User Interface and Screenshots 22

Chapter 8: Testing 25

Chapter 9: Results and Discussion 27

Chapter 10: Conclusion and Future Scope 28

References 29

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
DDL	Data Definition Language
ER	Entity–Relationship
FSM	Finite State Machine
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
MTTR	Mean Time to Resolution
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
SRS	Software Requirements Specification
TLS	Transport Layer Security
UI / UX	User Interface / User Experience

Chapter 1: Introduction

1.1 Background

A university hostel is a small community living under one roof. Hundreds or thousands of students share bathrooms, corridors, electrical fittings, water supply lines, furniture and network equipment. Constant use means that faults appear every day, and each one affects the comfort, health, safety or study routine of the residents. A leaking pipe, a flickering light or a broken door lock may look minor, but if it is not repaired quickly it can grow into a serious problem.

Fast repair depends on two things: the problem must reach the right person quickly, and someone must be clearly responsible for fixing it. In many hostels neither happens reliably, because the process for reporting and tracking faults is manual.

1.2 Problem Statement

Existing practices for handling hostel maintenance have the following weaknesses:

- **Paper registers:** Students must walk to the warden's desk or security gate to write a complaint. Entries are read hours or days later, handwriting may be unclear, pages can be lost or altered, and the student never learns what happened to the complaint.
- **Messaging groups:** Complaints posted in large chat groups are quickly buried. Nobody is formally assigned to them, personal details such as room numbers and phone numbers are visible to everyone, and there is no way to measure performance.
- **Large ERP portals:** Institutional ERP modules are too heavy for this job. Their long forms and desktop-oriented screens discourage students and maintenance staff from using them, so the data quickly goes stale.

There is therefore a need for a simple, mobile-friendly and low-cost system that records each complaint in a structured form, shows its status to the student, assigns responsibility to a named technician, keeps a history of every action, and gives the warden statistics to manage the maintenance workload.

1.3 Proposed Solution

Hostel Tracker is a web application that digitizes the full complaint lifecycle: lodging, assignment, repair, resolution, verification and feedback. Each user logs in with a role-specific

account and sees only the screens and actions meant for that role. A built-in analytics dashboard turns the stored records into counts, charts and averages.

1.4 Objectives

1. To allow any resident to lodge a structured complaint quickly from a phone or computer.
2. To give every complaint a unique, human-readable number (CMP-YYYY-XXXX) and a controlled status lifecycle.
3. To restrict features by role (resident, staff, admin) using token-based authentication.
4. To record every change to a complaint in an audit history, including who made it and when.
5. To let the warden assign complaints to staff and see each technician's workload.
6. To compute operational statistics such as complaint counts, resolution rate, average rating and average resolution time.
7. To collect resident ratings and feedback on completed work.
8. To publish hostel notices that expire automatically.
9. To deploy the system on free-tier cloud services so that it can run at no cost.

1.5 Scope and Limitations

The system covers complaint management, staff management, notices, analytics and audit history for a single hostel. It does not currently send SMS, e-mail or WhatsApp notifications, does not support file upload for images (an image URL can be stored instead), has no offline mode, and supports a single hostel rather than multiple campuses. These are listed as future work in Chapter 10.

1.6 Organization of the Report

Chapter 2 compares existing approaches. Chapter 3 lists the requirements. Chapter 4 describes the architecture and database design. Chapter 5 explains the implementation. Chapter 6 documents the API. Chapter 7 presents screenshots of the working system. Chapter 8 covers testing, Chapter 9 results and discussion, and Chapter 10 the conclusion and future scope.

Chapter 2: Literature Survey and Existing Systems

2.1 Categories of Existing Solutions

Before designing the system, the following existing approaches to maintenance and issue tracking were reviewed.

- **Computerized Maintenance Management Systems (CMMS):** Products such as Fiix, UpKeep and MaintainX are designed for factories and large facilities. They focus on asset registers, preventive maintenance schedules and spare-parts inventory. They are powerful but too complex and costly for a hostel, and they offer no simple interface for students.
- **IT helpdesk tools:** Products such as Jira Service Management, Zendesk and Freshdesk handle ticket queues well, but they are priced per agent and have no built-in idea of rooms, hostel blocks or maintenance trades. They would need heavy customization.
- **Institutional ERP modules:** These can record complaints, but their interfaces are long and difficult for students and field technicians, particularly on mobile phones.
- **Manual and informal methods:** Paper registers and messaging groups are free and familiar, but offer no tracking, privacy, audit trail or statistics.

2.2 Comparison

Table 2.1: Comparison of approaches

Feature	Paper register	Messaging group	CMMS / helpdesk	Hostel Tracker
Submission	In person	Free text	Web form	Mobile-friendly web form
Complaint ID	Page number	None	Generic ID	CMP-YYYY-XXXX
Status visible to student	No	No	Yes	Yes
Assigned technician	Verbal	No	Yes	Yes, with workload view
Audit trail	No	Chat history only	Yes	Yes, every change logged
Statistics	Manual	None	Yes	Dashboard and analytics
Resident feedback	No	Informal	Add-on	Built-in 1–5 rating
Notice board	Paper	Group message	Rarely	Digital, auto-expiring
Cost	Low	Free	High	Free-tier hosting

2.3 Gap Identified

None of the approaches above combines (a) a very simple student interface, (b) hostel-specific concepts such as room number and maintenance trade, (c) a full history of actions, and (d) near-zero running cost. Hostel Tracker is designed to fill this gap.

Chapter 3: Requirements Specification

3.1 Users and Roles

Table 3.1: User roles and permissions

Role	Main permissions
Resident	Register and log in; lodge complaints in 8 categories with 4 priority levels; track own complaints; view notices; rate and review resolved complaints.
Staff (technician)	Log in; view assigned complaints; move a complaint to “in progress” and “resolved”; add diagnosis or repair notes.
Admin (warden)	Full access: view all complaints; assign staff; change priority and status; add, edit and remove staff records; publish and retire notices; view analytics and the global maintenance history; delete complaints.

3.2 Functional Requirements

- **FR-1 Authentication:** Users register with username, e-mail, password, name, room and phone. Passwords are stored as bcrypt hashes. Login returns a signed JWT (valid for 7 days) containing the user id, username and role. Protected requests send the token as a Bearer header and invalid or expired tokens are rejected.
- **FR-2 Complaint lodging:** A complaint captures resident name, room number, phone, e-mail, category (plumbing, electrical, civil, internet, pest, furniture, hygiene, security), priority (low, medium, high, critical), title, description and an optional image URL. The system generates the complaint number and sets the status to “open”.
- **FR-3 Lifecycle control:** Status changes follow a fixed transition table (Section 4.4). When a complaint becomes resolved, the resolution time is stored.
- **FR-4 Audit history:** Every creation, assignment, status change, comment and rating adds a row to the maintenance history with the action, old and new values, the person who acted, notes and a timestamp.
- **FR-5 Staff management:** Admins manage staff name, role, specialization, contact details and active status. The staff list shows each person’s workload. Deleting a staff record must not delete past complaints.
- **FR-6 Analytics:** The system reports total, open, in-progress, resolved and critical counts, average rating, average resolution time, category and priority distributions, and a 14-day activity trend.

- **FR-7 Notices:** Admins post notices of type info, warning, maintenance or urgent with an optional expiry time. Expired notices are not shown to users.
- **FR-8 Feedback:** After a complaint is resolved, the resident can give a 1–5 star rating and written feedback.

3.3 Non-Functional Requirements

Table 3.2: Non-functional requirements

Attribute	Requirement
Performance	Normal API queries should respond in well under one second on the deployed system.
Security	bcrypt password hashing; signed JWT; parameterized SQL queries; role checks on protected routes; TLS for all connections.
Usability	Responsive layout that works on phone and desktop; dark and light themes.
Reliability	Relational database with foreign keys and CHECK constraints; keep-alive ping to reduce free-tier cold starts.
Maintainability	Separate route modules per feature; front end and back end deployed independently.
Cost	Run entirely on free-tier hosting.

3.4 Software and Hardware Requirements

Table 3.3: Environment

Item	Details
Front end	React 19, Vite, React Router, Recharts, Lucide icons
Back end	Node.js, Express.js, jsonwebtoken, bcryptjs, pg, dotenv, cors
Database	PostgreSQL on Neon (serverless)
Hosting	Vercel (front end), Render (back end)
Development machine	Any computer with Node.js 18+ and a modern browser; 4 GB RAM or more
Client	Any modern browser on phone, tablet or computer

Chapter 4: System Design

4.1 Architecture

Hostel Tracker follows a three-tier architecture. The browser runs a React single-page application that talks to a stateless Express REST API over HTTPS. The API reads and writes data in a PostgreSQL database hosted on Neon. Because the API keeps no session state (the JWT carries the identity), it can be restarted or scaled without losing user sessions.

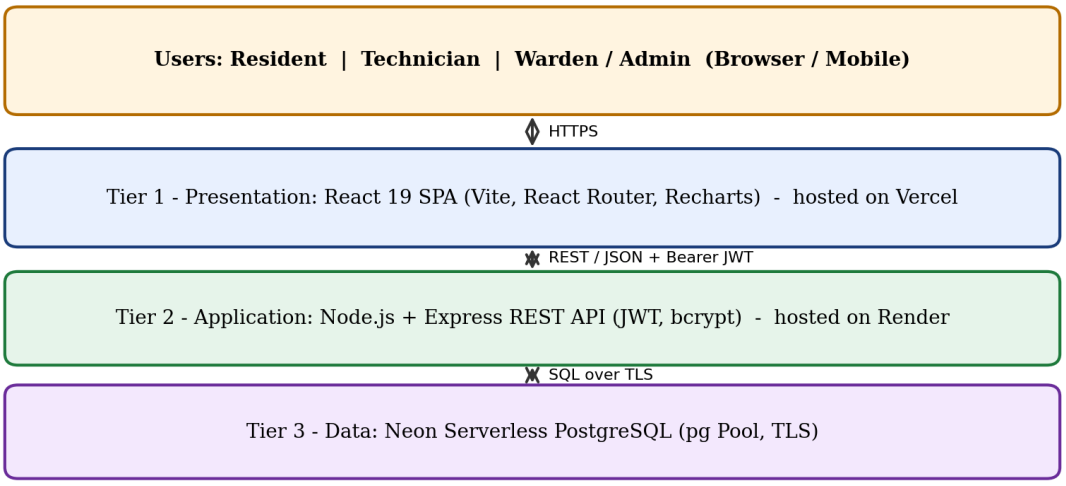


Figure 4.1: Three-tier architecture of Hostel Tracker

4.2 Data Flow for Lodging a Complaint

When a resident submits the “New Complaint” form, the following steps occur:

1. The browser sends POST /api/complaints with the form data and the Bearer token.
2. The server validates that title, room number, category and description are present; otherwise it returns 400.
3. The server finds the highest complaint number for the current year and increments it to create the next CMP-YYYY-XXXX number.
4. The server inserts the complaint with status “open” using a parameterized query.
5. The server inserts a “created” row in maintenance_history.
6. The server returns 201 Created with the new complaint, and the interface shows a success message.

4.3 Database Design

The database has five tables. The design is in Third Normal Form: staff details are stored only in the staff table, and complaints refer to staff and users through foreign keys, so a change to a technician's phone number is made in one place.

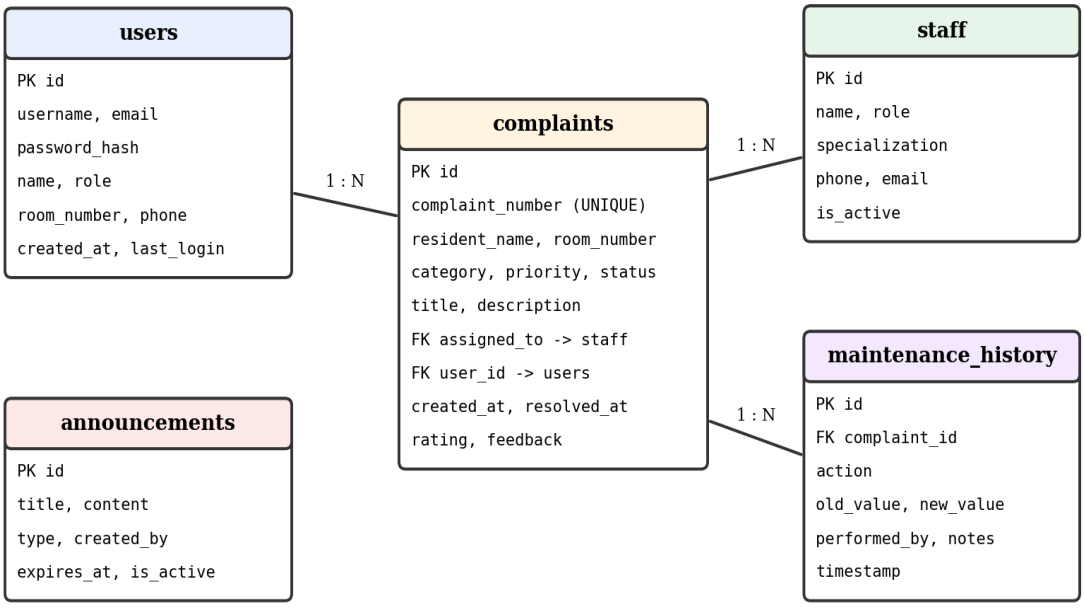


Figure 4.2: Entity–relationship diagram

Table 4.1: Database tables

Table	Purpose and key points
users	Login identity and profile. Role restricted to resident, staff or admin. Username and e-mail are unique.
staff	Maintenance personnel directory with specialization, contact details and an active flag.
complaints	Core work orders. Unique complaint_number; CHECK constraints on priority, status and rating (1–5). assigned_to references staff (ON DELETE SET NULL); user_id references users (ON DELETE SET NULL).
maintenance_history	Audit trail. complaint_id references complaints (ON DELETE CASCADE). Stores action, old_value, new_value, performed_by, notes and timestamp.
announcements	Notices with type, creator, optional expires_at and is_active flag.

Indexes are created on complaint status, category, priority, user_id and assigned_to, on the history complaint_id, and on the notice active/expiry columns to speed up the common filters.

4.4 Complaint Lifecycle (Finite State Machine)

A complaint can only move between statuses along allowed transitions. This prevents unlogged or illogical changes, such as closing a complaint that was never worked on.

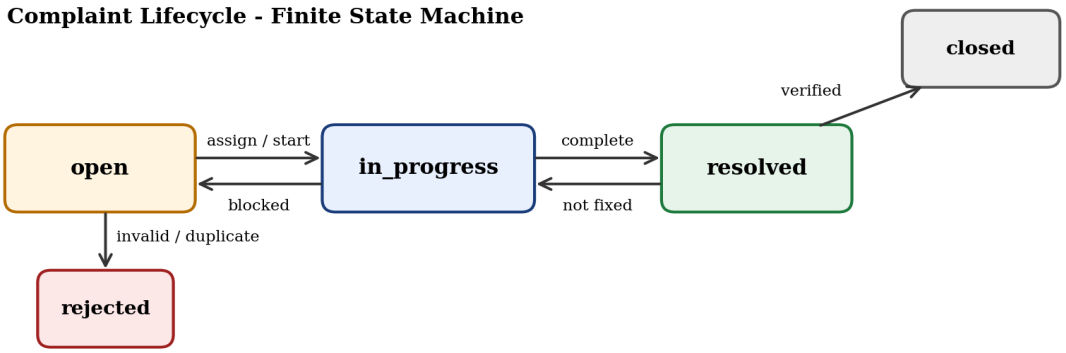


Figure 4.3: Complaint status lifecycle

Table 4.2: Allowed status transitions

From	To	Meaning
open	in_progress	Staff assigned and work started
open	rejected	Invalid or duplicate complaint
in_progress	resolved	Technician completed the repair
in_progress	open	Work blocked; assignment removed
resolved	closed	Resident verified the repair
resolved	in_progress	Issue not actually fixed; work reopened
closed / rejected	—	Terminal states

4.5 Security Design

- Passwords are hashed with bcrypt before being stored; plain passwords are never saved.
- Login issues an HMAC-SHA256 signed JWT. The secret is kept in an environment variable and never committed to source control.
- All database access uses parameterized queries ($1, $2, …), which prevents SQL injection.
- Role checks on the server decide who may assign staff, delete records or manage notices.
- The database connection uses TLS.

Chapter 5: Implementation

5.1 Project Structure

```
hostel-tracker/
  backend/
    server.js            Express app, routes, keep-alive
    database/db.js       pg Pool, schema creation, query helpers
    routes/
      auth.js            register, login, profile
      complaints.js      CRUD, status, assignment, rating, comments
      staff.js           staff directory and workload
      analytics.js       summary, trends, resolution time
      announcements.js   notices with expiry
  frontend/
    src/
      pages/             Dashboard, ComplaintsList, ComplaintDetail,
                         NewComplaint, MaintenanceHistory, Analytics,
                         Staff, Announcements, Settings, AuthPage
      components/        Sidebar, header, shared UI
      context/           AuthProvider, AppProvider (toasts)
```

5.2 Back-End Modules

- **Database layer (db.js):** Creates a pg connection pool (maximum 20 connections, 30-second idle timeout) and provides query, queryAll and queryOne helpers. On start-up it creates all tables if they do not exist.
- **Server (server.js):** Configures CORS and JSON parsing, exposes a /api/health endpoint, mounts the five route modules, and installs a global error handler.
- **Keep-alive:** Free hosting platforms put idle services to sleep. In production the server pings its own /api/health endpoint every 13 minutes to avoid slow cold starts.

5.3 Complaint Number Generation

Each complaint receives a number of the form CMP-YYYY-XXXX. The server looks up the latest number for the current year, adds one, and pads it to four digits. The numbering restarts at 0001 each year.

```
SELECT complaint_number FROM complaints
WHERE complaint_number LIKE 'CMP-2026-%'
ORDER BY id DESC LIMIT 1;

-- no row   -> CMP-2026-0001
-- else     -> split by "-", add 1 to last part, left-pad to 4 digits
```

Note: if two complaints were submitted at exactly the same moment, they could read the same “last” number. The UNIQUE constraint on complaint_number would then reject one insert. A retry on conflict or a database sequence would remove this risk and is noted as an improvement.

5.4 Audit History

Whenever a complaint is created, assigned, changed in status or priority, commented on or rated, the server inserts a row into maintenance_history. This history is shown as a timeline on the complaint detail page and as a global list on the Maintenance History page. Because rows are only ever inserted, the history shows the full story of each complaint.

5.5 Analytics and Mean Time to Resolution

Average resolution time is calculated from the complaints that have a resolved_at timestamp:

```
SELECT ROUND(AVG(EXTRACT(EPOCH FROM (resolved_at - created_at)) / 3600)::numeric,
1)
       AS avg_resolution_hours
FROM complaints
WHERE resolved_at IS NOT NULL
  AND status IN ('resolved', 'closed');
```

Other summary values (counts by status, category and priority; average rating) are computed with COUNT, AVG and GROUP BY queries. The 14-day trend counts complaints created and resolved per day.

5.6 Staff Workload

The staff page shows, for each technician, the number of complaints assigned, the number still active (open or in progress) and the number resolved. This helps the warden assign new work to the person with the lowest active load in the required trade.

5.7 Notice Board

Notices are stored with an optional expires_at time. The public notices query returns only active notices whose expiry is empty or in the future (expires_at > NOW()), so expired notices disappear without any manual clean-up.

5.8 Front-End Implementation

The front end is a React 19 single-page application built with Vite. React Router handles page navigation without full reloads. An AuthProvider stores the logged-in user and token and attaches the token to API calls; an AppProvider displays toast notifications. Pages show data fetched from the API, and Recharts draws the status and trend charts. The visual design uses CSS custom properties for colours, which makes the dark and light themes possible. The layout adapts from small phone screens to wide desktop screens.

5.9 Deployment

The front end is built with Vite and hosted on Vercel, with a rewrite rule so that all routes load the single-page app. The back end runs on Render and is connected to the Neon PostgreSQL database through a DATABASE_URL environment variable. Secrets are set as environment variables on the hosting platforms and are not stored in the repository.

Table 5.1: Environment variables

Variable	Purpose
PORT	Port for the Express server
NODE_ENV	Set to production on the hosted server
DATABASE_URL	Neon PostgreSQL connection string (with SSL)
JWT_SECRET	Long random secret used to sign tokens (never share or commit)
RENDER_EXTERNAL_URL	Public URL used by the keep-alive ping
VITE_API_URL	Back-end API base URL used by the front end

Chapter 6: REST API

All endpoints exchange JSON. Protected endpoints require the header Authorization: Bearer <token>.

Table 6.1: API endpoints

Method	Path	Access	Purpose
POST	/api/auth/register	Public	Create an account
POST	/api/auth/login	Public	Log in and receive a JWT
GET	/api/auth/me	Token	Get the current user
PATCH	/api/auth/profile	Token	Update phone / room
GET	/api/complaints	Token	List and filter complaints
POST	/api/complaints	Token	Lodge a complaint
GET	/api/complaints/:id	Token	Complaint with history
PATCH	/api/complaints/:id	Token	Update status, assignment, priority, rating
DELETE	/api/complaints/:id	Admin	Delete a complaint
POST	/api/complaints/:id/comment	Token	Add a comment
GET	/api/analytics/summary	Token	Counts, averages, distributions
GET	/api/analytics/trends	Token	14/30-day trend
GET	/api/staff	Token	Staff list with workload
POST/PATCH/DELETE	/api/staff[/:id]	Admin	Manage staff
GET	/api/announcements	Token	Active notices
POST/PATCH/DELETE	/api/announcements[/:id]	Admin	Manage notices
GET	/api/history	Token	Global maintenance history
GET	/api/health	Public	Health check / keep-alive

Example request body for creating a complaint:

```
POST /api/complaints
{
  "resident_name": "Arjun Mehta",
  "room_number": "A-101",
  "phone": "9811234567",
  "category": "plumbing",
  "priority": "high",
  "title": "Leaking tap in bathroom",
  "description": "Water is leaking continuously from the basin tap."
}

201 Created -> { "id": 1, "complaint_number": "CMP-2026-0001",
                 "status": "open", ... }
```

Note: access levels in this table describe the intended design. Where the current code leaves a route open to unauthenticated users (for example, read-only analytics), tightening it to require a token is recommended before production use.

Chapter 7: User Interface and Screenshots

This chapter presents screenshots of the deployed application. The live front end is hosted on Vercel and uses the Neon database (the green “Neon DB” indicator in the header shows the database connection).

7.1 Sign In / Sign Up Page

The entry page offers Sign In and Sign Up tabs. Users can sign in with a username or e-mail and a password (with a show/hide control). For demonstration, three one-click demo logins are provided: Admin Warden, Resident (Arjun) and Staff (Rajesh). A theme toggle in the top-right corner switches between dark and light modes.

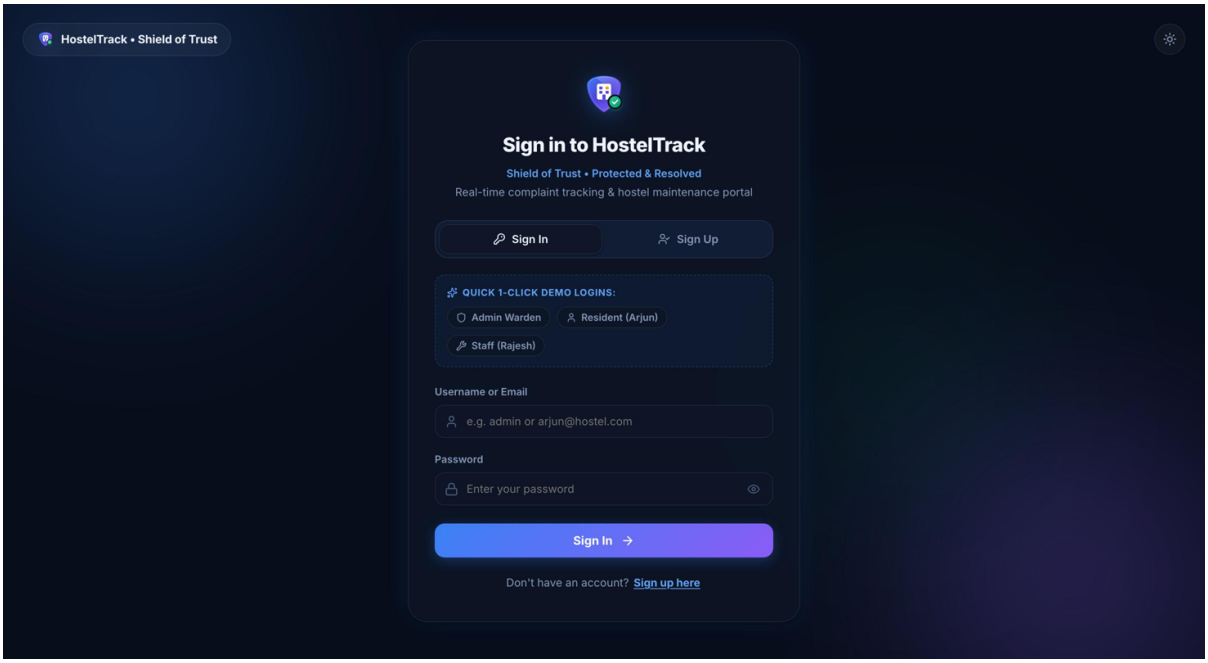


Figure 7.1: Sign-in page with demo logins

7.2 Dashboard

After login, the warden sees the dashboard. The sidebar groups navigation into Overview (Dashboard, Analytics), Complaints (All Complaints, New Complaint) and Management (Maintenance History, Staff, Announcements, Settings). The main area shows KPI cards: total complaints, open, in progress, resolved, critical, and average rating. Below them are an activity chart for the last 14 days and a status breakdown donut chart with the average resolution time. The “New Complaint” button is available at the top right.



Figure 7.2: Admin dashboard with KPI cards and status breakdown

The dashboard in Figure 7.2 shows 12 complaints in total: 5 open, 4 in progress and 3 resolved, with 3 marked critical and an average resident rating of 3.67 stars. The 14-day activity chart shows “No trend data yet” because the sample complaints are dated outside the most recent 14-day window.

7.3 All Complaints

The complaints page lists every complaint in a table with the complaint number, title and resident name, room, category badge, priority badge (Low, Medium, High, Critical), status badge (Open, In Progress, Resolved), assigned staff member (or “Unassigned”) and date. Each row has a view button to open the detail page and a delete button for admins. The red badge beside “All Complaints” in the sidebar shows the number of open items that need attention.

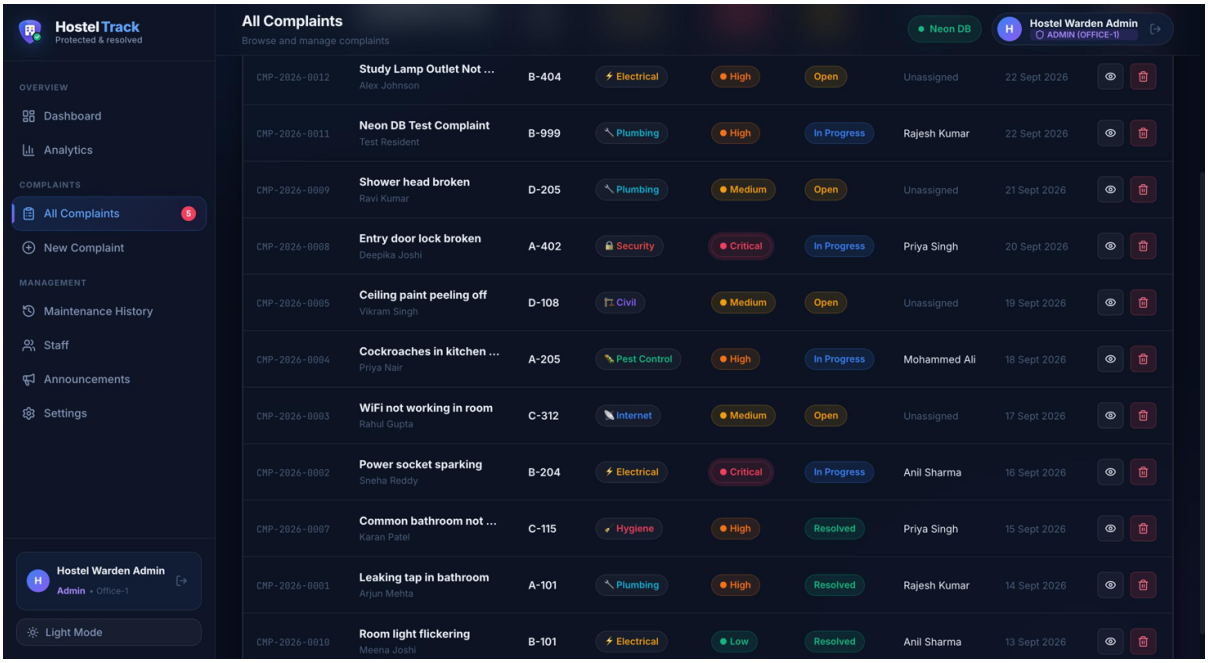


Figure 7.3: All Complaints list with priority, status and assignment

7.4 Other Screens

- New Complaint: form for category, priority, title, description and contact details.
- Complaint Detail: full information, assignment and status controls, comments, rating, and a timeline of the audit history.
- Maintenance History: global list of all logged actions.
- Analytics: charts for category, priority and status distribution, resolution time and staff performance.
- Staff: technician cards with specialization, contact details and workload.
- Announcements: color-coded notices with expiry.
- Settings: profile and preference options.

Chapter 8: Testing

8.1 Testing Approach

The system was tested at three levels: unit checks of helper logic (complaint number generation, token verification, date formatting), integration checks of the API against the PostgreSQL database, and security checks (SQL-injection attempts, tampered tokens, role restrictions). Manual end-to-end testing was done on the deployed application using the three demo accounts.

8.2 Test Cases

Table 8.1: Test cases

ID	Feature	Input / action	Expected result	Result
T-01	Registration	Valid resident details	Account created; JWT returned (201)	Pass
T-02	Registration	Duplicate username	409 Conflict	Pass
T-03	Login	Wrong password	401 Unauthorized	Pass
T-04	Auth middleware	No token	401 returned	Pass
T-05	Auth middleware	Tampered token	403 returned	Pass
T-06	Complaint number	New complaint after CMP-2026-0001	CMP-2026-0002 generated	Pass
T-07	Validation	Title missing	400 Bad Request	Pass
T-08	Lodging	Priority “critical”	Saved and flagged critical (201)	Pass
T-09	Assignment	Assign to staff #2	assigned_to updated; history row added	Pass
T-10	Lifecycle	open → in_progress	Status updated; history row added	Pass
T-11	Lifecycle	→ resolved	resolved_at set	Pass
T-12	Audit trail	GET complaint by id	Record with history array	Pass
T-13	Rating	5 stars on resolved complaint	Rating saved; “rated” logged	Pass
T-14	Rating	Rating of 6	Rejected by CHECK constraint	Pass
T-15	Notices	Expired notice	Not returned in active list	Pass
T-16	Security	Search text ' OR '1'='1	Treated as plain text; no data leak	Pass
T-17	Integrity	Delete assigned staff	Complaints kept; assigned_to = NULL	Pass

ID	Feature	Input / action	Expected result	Result
T-18	Integrity	Delete complaint	History rows deleted (cascade)	Pass
T-19	Analytics	GET summary	Correct counts, rating, average time	Pass

Chapter 9: Results and Discussion

9.1 Results from the Deployed System

The sample data loaded in the deployed system (visible in Figures 7.2 and 7.3) gives the following results:

Table 9.1: Dashboard figures from the deployed system

Measure	Value
Total complaints	12
Open	5
In progress	4
Resolved	3 (25% resolution rate)
Critical priority	3
Average resident rating	3.67 / 5
Average resolution time	24 hours (shown on dashboard)

The complaint list shows the system handling all categories (electrical, plumbing, civil, internet, pest control, hygiene, security) and all priority levels. Critical complaints such as the sparking power socket (CMP-2026-0002) and the broken entry door lock (CMP-2026-0008) are both already in progress, while several medium-priority complaints remain unassigned, which the warden can see at a glance and act on.

9.2 Discussion

- **Strengths:** The system gives a single, clear place to report and follow complaints. Roles keep each user's screen simple. The audit history provides accountability, and the dashboard replaces manual counting. Free-tier hosting keeps the running cost at zero.
- **Limitations:** The data shown is demonstration data from a small test deployment, so the figures are illustrative and not a field study; a controlled pilot in a real hostel would be needed to measure the improvement over paper-based handling. The free hosting tiers can still respond slowly after long idle periods, no notifications are sent, and an internet connection is required.
- **Known improvements:** Use a database sequence or a retry for complaint numbers; require authentication on all analytics routes; add server-side input length limits and rate limiting; and use image upload instead of image URLs.

Chapter 10: Conclusion and Future Scope

10.1 Conclusion

Hostel Tracker replaces paper registers and informal chat groups with a structured, trackable and measurable maintenance process. Residents can lodge and follow complaints from any device, wardens can assign and monitor work, and technicians can update progress in a few taps. The use of a three-tier architecture, role-based JWT authentication, a normalized PostgreSQL schema, a controlled status lifecycle and a full audit history makes the system dependable, while deployment on free-tier cloud services keeps it affordable for any institution.

The project achieved its objectives: structured complaint lodging, unique complaint numbers, role-based access, an audit trail, staff assignment with workload visibility, analytics, feedback collection and an expiring notice board.

10.2 Future Scope

1. SMS, e-mail or WhatsApp notifications when a complaint changes status.
2. Image upload for photographs of faults, stored in cloud storage.
3. Progressive Web App (PWA) with offline drafting of complaints.
4. Automatic suggestion of the best technician based on trade and current workload.
5. Predictive analysis of recurring faults using historical data.
6. Multi-hostel and multi-campus support with tenant identifiers.
7. Rate limiting, stricter validation and automated test suites for continuous integration.

References

1. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2015.
2. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill, 2020.
3. R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
4. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. https://datatracker.ietf.org/doc/html/rfc7519
5. React Documentation. https://react.dev/
6. Express.js Documentation. https://expressjs.com/
7. PostgreSQL Documentation. https://www.postgresql.org/docs/
8. Neon Serverless Postgres Documentation. https://neon.tech/docs
9. Vite Documentation. https://vite.dev/
10. Recharts Documentation. https://recharts.org/

Github - https://github.com/aryanprajap/Task-Manager.git